AI ASSISTED CODING

Lab Assignment-13.4

Name : T.Shylasri

H.T.NO: 2303A51876

Batch-14(LAB-13)

Date: 12-03-2026

Lab 13: Code Refactoring – Improving Legacy Code with AI Suggestions

Lab Objectives

• Identify code smells and inefficiencies in legacy Python scripts.

• Use AI-assisted coding tools to refactor for readability, maintainability, and performance.

• Apply modern Python best practices while ensuring output correctness.

Learning Outcome

After completing this assignment, students will be able to:

• Identify refactoring opportunities in legacy Python code

• Apply AI-assisted suggestions to refactor code without changing behavior

• Refactor Python code using Pythonic and scalable constructs

• Critically evaluate AI-generated refactoring recommendations

Task 1: Refactoring Data Transformation

Logic

Scenario

You are maintaining a data preprocessing script where numerical transformations are written using verbose loops.

Task Description

• Review the legacy code that computes transformed values using an explicit loop.

• Use an AI tool to suggest a more Pythonic refactoring approach.

• Refactor the code using list comprehensions or helper functions while preserving the output.

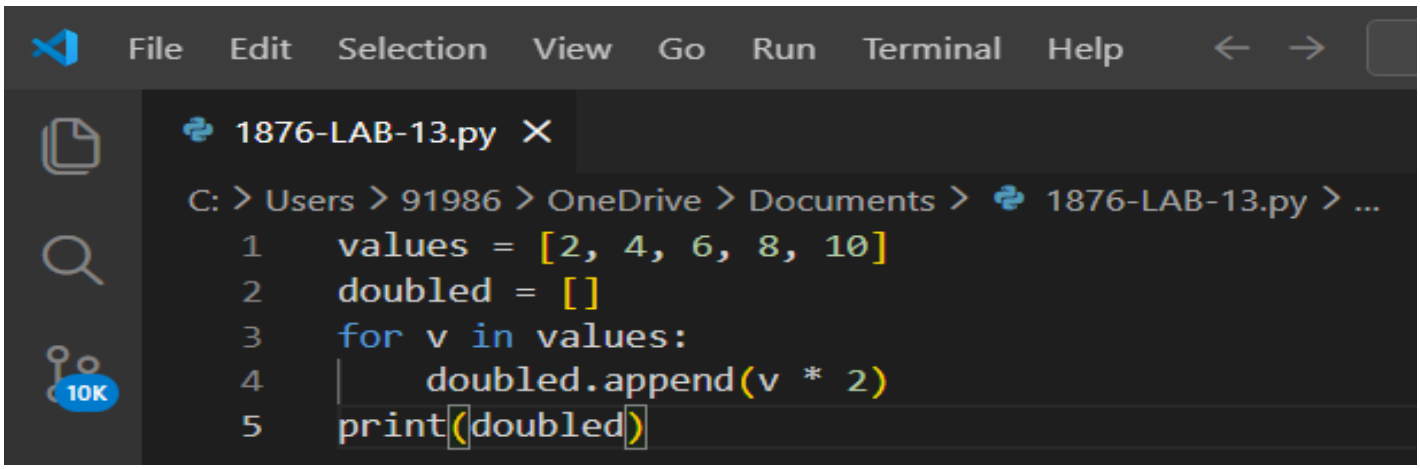
Legacy Code

```
values = [2, 4, 6, 8, 10]

doubled = []

for v in values:

        doubled.append(v * 2)

print(doubled)
```

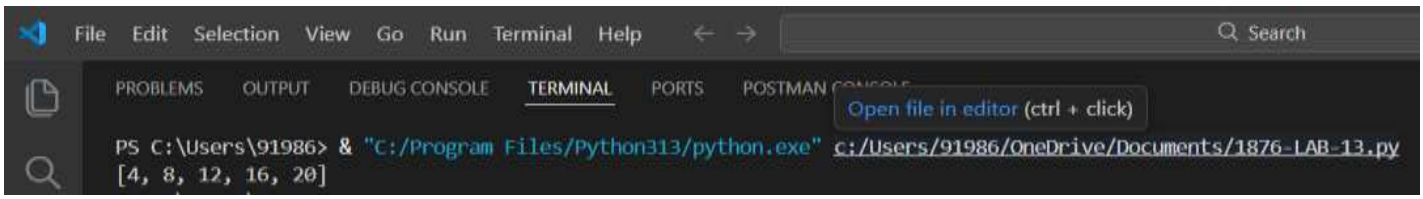
Expected Output

[4, 8, 12, 16, 20]

Legacy Code



Output



AI Prompt for Refactored Code

#Refactor the loop-based code into a more Pythonic version using list comprehension and modify it to take user input while keeping the same output.

Prompt Screenshot

Refactored Code

```python
#Refactor the loop-based code into a more Pythonic version using list comprehension and modify it to take user input while keeping the same output.
user_input = input("Enter numbers separated by space: ")
values = list(map(int, user_input.split()))
doubled = [v * 2 for v in values]
print("Doubled values:", doubled)
```

Output

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS    POSTMAN CONSOLE

PS C:\Users\91986> & "C:/Program Files/Python313/python.exe" c:/Users/91986/OneDrive/Documents/1876-LAB-13.py
Enter numbers separated by space: 2 4 6 8 10
Doubled values: [4, 8, 12, 16, 20]
```

Algorithm

Start the program

1. Ask the user to enter numbers separated by spaces
2. Read the input as a string
3. Convert the string into a list of integers
4. Apply list comprehension to multiply each number by 2
5. Store the transformed values in a new list
6. Print the doubled values
7. End the program

Observation

The legacy Python code uses an explicit loop and the append() method to double each value in the list. While it works correctly, the code is longer and less efficient. After refactoring, list comprehension was used, which made the code shorter, clearer, and easier to read while still producing the same output.

Explanation

List comprehension is a Python feature that allows creating a new list by applying an operation to each element in an existing list in a single line. Instead of using a loop and append(), the transformation v * 2 is directly applied within the list comprehension. The program was also modified to accept input values from the user.

Justification

The refactored code is preferred because it follows Python best practices. It reduces the number of lines of code, improves readability, and makes the program easier to maintain. The functionality remains unchanged, ensuring that the output is still correct.

Conclusion

The legacy code was successfully refactored using list comprehension with the help of AI suggestions. The new implementation is more Pythonic, efficient, and maintainable while preserving the original program behavior.

Task 2: Improving Text Processing Code Readability

Scenario

You are working on a log message generator that builds messages word by word.

Task Description

• Analyze the legacy code that constructs a sentence using repeated string concatenation.

• Ask AI to suggest a more efficient and readable approach.

• Refactor the code accordingly while keeping the final output unchanged.

Legacy Code

words = ["Refactoring", "with", "AI", "improves", "quality"]

message = ""

for w in words:

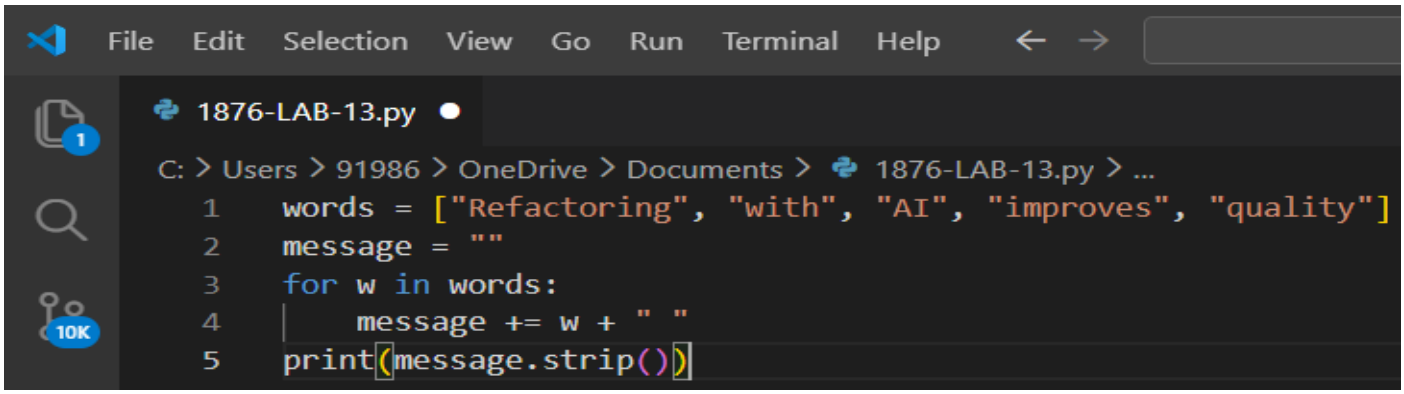
    message += w + " "

print(message.strip())

Expected Output

Refactoring with AI improves quality

Legacy Code



Problem with Legacy Code

- Uses **repeated string concatenation**
- Less **efficient for large text**

- Code readability can be improved
- Not the **best Python practice**

AI Prompt for Refactored Code

#Refactor the code that concatenates strings in a loop into a more efficient and readable approach using Python best practices,and modify it to take user input while keeping the same output.

Prompt Screenshot

Refactored Code

Output

Algorithm

Start the program

1. Ask the user to enter words separated by spaces
2. Read the input as a string
3. Split the string into a list of words
4. Use the join() method to combine the words into a sentence
5. Print the generated message
6. End the program

Observation

The legacy code creates a sentence by repeatedly adding words using string concatenation inside a loop. Although the code works correctly, it is not the most efficient method. After refactoring, the join() method is used, which makes the code shorter and more readable while producing the same output.

Explanation

The join() method in Python combines all elements of a list into a single string using a specified separator. Instead of adding each word one by one in a loop, join() merges the words in a single operation. The program also allows the user to enter words as input.

Justification

The refactored code is better because it follows Python best practices. Using join() is more efficient than repeated string concatenation and improves code readability and maintainability without changing the program's output.

Conclusion

The legacy text-processing code was successfully refactored using the join() method with the help of AI suggestions. The new code is more efficient, readable, and Pythonic while maintaining the same functionality.

Task 3: Safer Access to Configuration Data

Scenario

You are maintaining a configuration loader where dictionary keys may

or may not exist depending on deployment settings.

Task Description

- Review the legacy code that manually checks for dictionary keys.
- Use AI suggestions to refactor the code using safer dictionary access methods.
- Ensure the behavior remains the same for missing keys.

Legacy Code

```
config = {"host": "localhost", "port": 8080}

if "timeout" in config:

        print(config["timeout"])

else:

print("Default timeout used")
```

Expected Output

Default timeout used

Legacy Code

```
1876-LAB-13.py
C: > Users > 91986 > OneDrive > Documents > 1876-LAB-13.py > ...
config = {"host": "localhost", "port": 8080}
if "timeout" in config:
    print(config["timeout"])
else:
    print("Default timeout used")
```

Output

```
PROBLEMS   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS   POSTMAN   Open file in editor (ctrl + click)
PS C:\Users\91986> & "C:/Program Files/Python313/python.exe" c:/Users/91986/OneDrive/Documents/1876-LAB-13.py
Default timeout used
```

Problem with Legacy Code

- Uses manual key checking
- More lines of code
- Less readable
- Can be simplified using safer dictionary methods

AI Prompt for Refactoring Code

Refactor the code that manually checks dictionary keys into a safer and more Pythonic approach using dictionary methods while keeping the same behavior and allowing user input.

Prompt Screenshot

```
1876-LAB-13.py
C: > Users > 91986 > OneDrive > Documents > 1876-LAB-13.py
## Refactor the code that manually checks dictionary keys into a safer and
# more Pythonic approach using dictionary methods while keeping the same behavior and allowing user input.
```

Refactored Code

```python
## Refactor the code that manually checks dictionary keys into a safer and
# more Pythonic approach using dictionary methods while keeping the same behavior and allowing user input.
host = input("Enter host: ")
port = int(input("Enter port: "))
timeout = input("Enter timeout (press Enter to skip): ")
config = {"host": host, "port": port}
if timeout:
    config["timeout"] = int(timeout)
value = config.get("timeout")
if value:
    print(value)
else:
    print("Default timeout used")
```

Output

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS    POSTMAN CONSOLE

PS C:\Users\91986> & "C:/Program Files/Python313/python.exe" c:/Users/91986/OneDrive/Documents/1876-LAB-13.py
Enter host: localhost
Enter port: 8080
Enter timeout (press Enter to skip):
Default timeout used
```

Algorithm

1. Start the program
2. Ask the user to enter host and port
3. Optionally ask the user to enter timeout
4. Store values in a dictionary
5. Use the get() method to safely access the timeout key
6. If timeout exists, print its value
7. Otherwise print “Default timeout used”
8. End the program

Observation

The legacy code checks whether a key exists in the dictionary before accessing it. This makes the code longer and slightly harder to read. After refactoring, the get() method is used to safely access dictionary values, which simplifies the code while producing the same output.

Explanation

The get() method in Python allows safe access to dictionary values without raising an error if the key does not exist. It returns None or a default value when the key is missing. This removes the need for manual key checking.

Justification

The refactored code is better because it reduces the number of lines and improves readability. Using get() follows Python best practices for handling optional dictionary keys and prevents potential errors when keys are missing.

Conclusion

The legacy configuration code was successfully refactored using the dictionary get() method. The new implementation is safer, more readable, and easier to maintain while keeping the same program behavior when keys are missing.

Task 4: Refactoring Conditional Logic for Scalability

Scenario

You are enhancing a utility module that performs different operations -based on user input.

Task Description

- Examine the multiple if–elif conditions used to determine operations.
- Ask AI to suggest a cleaner, scalable alternative.
- Refactor the logic using mapping techniques while preserving functionality.

Legacy Code

```
action = "divide"

x, y = 10, 2

if action == "add":

        result = x + y

elif action == "subtract":

        result = x - y

elif action == "multiply":

        result = x * y

elif action == "divide":
```

```
        result = x / y
else:
        result = None
print(result)
```

Expected Output

5.0

Legacy Code

```python
action = "divide"
x, y = 10, 2
if action == "add":
    result = x + y
elif action == "subtract":
    result = x - y
elif action == "multiply":
    result = x * y
elif action == "divide":
    result = x / y
else:
    result = None
print(result)
```

Output

```
PROBLEMS   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS   POSTMAN CONSOLE

PS C:\Users\91986> & "C:/Program Files/Python313/python.exe" c:/Users/91986/OneDrive/Documents/1876-LAB-13.py
5.0
```

Problem with Legacy Code

- Uses multiple if–elif conditions
- Difficult to extend if more operations are added
- Less scalable and harder to maintain

AI Prompt for Refactored Code

#Refactor the code with multiple if-elif conditions into a cleaner and scalable approach using a dictionary mapping technique while allowing user input and keeping the same functionality.

Prompt Screenshot

```
#Refactor the code with multiple if-elif conditions into a cleaner and scalable approach using a dictionary mapping technique
#while allowing user input and keeping the same functionality.
```

Refactored Code

```
#Refactor the code with multiple if-elif conditions into a cleaner and scalable approach using a dictionary mapping technique
#while allowing user input and keeping the same functionality.
action = input("Enter operation (add, subtract, multiply, divide): ")
x = float(input("Enter first number: "))
y = float(input("Enter second number: "))
operations = {
    "add": x + y,
    "subtract": x - y,
    "multiply": x * y,
    "divide": x / y
}
result = operations.get(action)
print(result)
```

Output

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS    POSTMAN CONSOLE

PS C:\Users\91986> & "C:/Program Files/Python313/python.exe" c:/Users/91986/OneDrive/Documents/1876-LAB-13.py
Enter operation (add, subtract, multiply, divide): divide
Enter first number: 10
Enter second number: 2
5.0
```

Algorithm (Step-by-Step)

1. Start the program
2. Ask the user to enter the operation
3. Ask the user to enter two numbers
4. Store available operations in a dictionary
5. Use the operation name as the key to find the corresponding operation
6. Perform the calculation
7. Display the result
8. End the program

Observation

The legacy code uses multiple if–elif conditions to determine the operation. This makes the program longer and harder to extend. After refactoring, a dictionary mapping technique is used, which simplifies the logic and makes the code cleaner.

Explanation

A mapping technique uses a dictionary where keys represent actions and values represent operations or functions. Instead of checking each condition using if–elif, the program directly retrieves the required operation from the dictionary based on the user input.

Justification

The refactored approach improves scalability and readability. Adding new operations becomes easier because we only need to add a new entry in the dictionary instead of modifying multiple conditional statements.

Conclusion

The conditional logic in the legacy code was successfully refactored using a dictionary mapping technique. This approach makes the program more scalable, readable, and easier to maintain while keeping the same functionality and output.

Task 5: Simplifying Search Logic in Collections

Scenario

You are reviewing legacy inventory-check code that uses manual loops

to find elements.

Task Description

• Identify the explicit loop used for searching an item.

• Use AI assistance to refactor the logic into a more concise and readable form.

• Maintain the same output behavior.

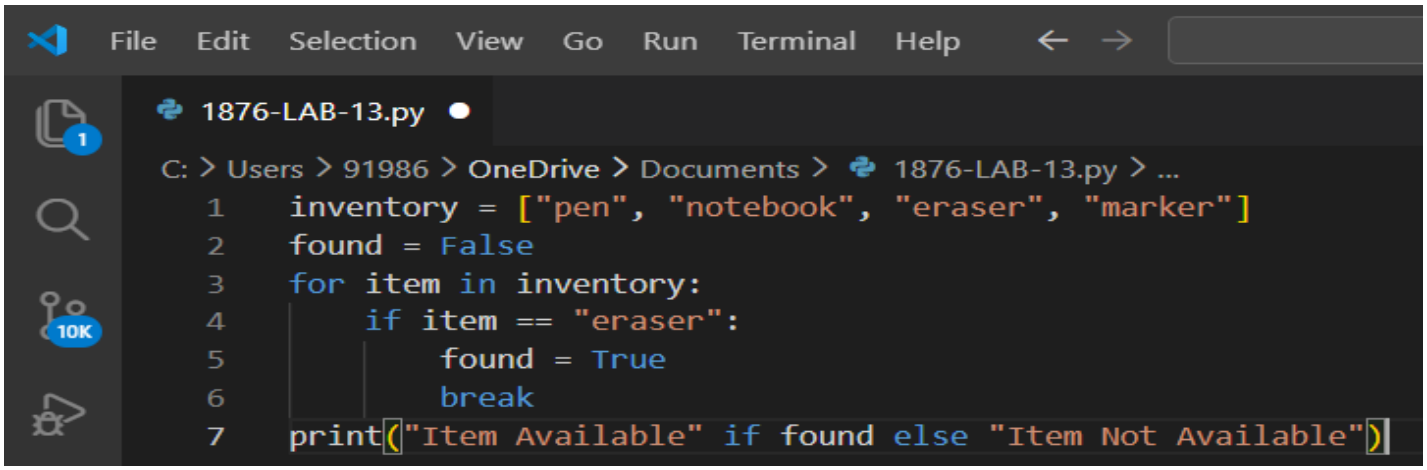
Legacy Code

```
inventory = ["pen", "notebook", "eraser", "marker"]

found = False

for item in inventory:

        if item == "eraser":

                found = True

                break

print("Item Available" if found else "Item Not Available")
```

Expected Output

Item Available

Legacy Code



Problem with Legacy Code

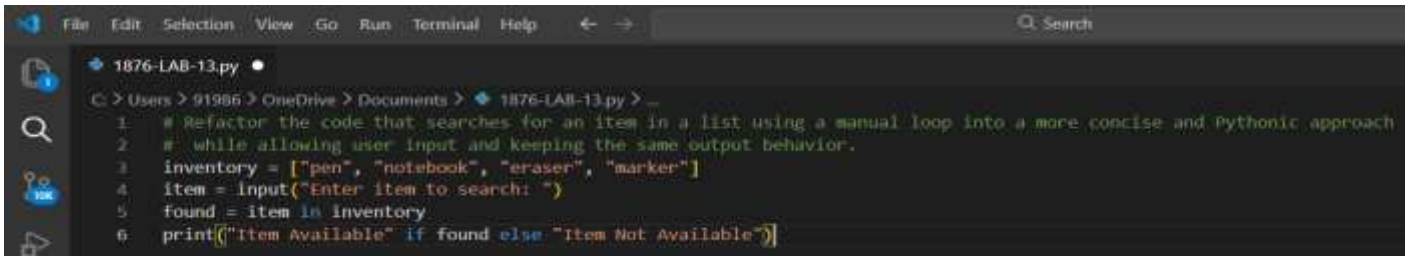
- Uses a manual loop for searching
- More lines of code
- Can be simplified using built-in Python operators
- Less readable compared to Pythonic approaches

AI Prompt for Refactored Code

Refactor the code that searches for an item in a list using a manual loop into a more concise and Pythonic approach while allowing user input and keeping the same output behavior.

Prompt Screenshot

Refactored Code



Output

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS    POSTMAN CONSOLE

PS C:\Users\91986> & "C:/Program Files/Python313/python.exe" c:/Users/91986/OneDrive/Documents/1876-LAB-13.py
Enter item to search: eraser
Item Available
```

Algorithm (Step-by-Step)

1. Start the program
2. Create a list containing inventory items
3. Ask the user to enter the item to search
4. Use the in operator to check whether the item exists in the list
5. If the item exists, display "Item Available"
6. Otherwise display "Item Not Available"
7. End the program

Observation

The legacy code searches for an item in the inventory using a manual loop and a flag variable. This approach works but requires more lines of code. After refactoring, the Python in operator is used, making the code shorter and easier to understand.

Explanation

The in operator in Python checks whether a value exists in a list or collection. Instead of iterating through each element manually, Python directly checks membership in a single expression.

Justification

The refactored code improves readability and reduces complexity. Using the in operator follows Python best practices and eliminates the need for additional variables and loops.

Conclusion

The legacy inventory search code was successfully refactored using the Python in operator. The new solution is more concise, readable, and efficient while maintaining the same output behavior.